

MINI PROJECT 5

Hotel Booking System

Database Management System Report

Group 9 - Mini Project 5

Himanshu Katiyara

Asad Jamil Antule

Sahil Kumar

Sasanka SekharKundu

PostgreSQL implementation using psql

Submitted as a practical demonstration of ER modelling, relational schema design, joins, JSONB, transactions, SAVEPOINT, and date/time operations.

1. Project Overview

The Hotel Booking System models a hotel chain in which hotels contain rooms, guests create bookings for date ranges, and bookings can have one or more payments. The implementation was carried out in PostgreSQL using the **hotel_booking** database. The attached terminal screenshots document table creation, sample data insertion, query execution, transaction handling, and final results.

Entities: Hotel, Room, Guest, Booking, Payment.

Core relationships: Hotel 1:M Room; Guest 1:M Booking; Room 1:M Booking; Booking 1:M Payment.

2. Part A — ER Model & Database Design

The ER structure is represented below with the required cardinalities and participation. Participation is stated from the business rules in the project specification.

Relationship	Cardinality	Participation	Meaning
Hotel — Room	1 : M	Hotel: Total / Room: Total	Each room belongs to one hotel; a hotel can have multiple rooms
Guest — Booking	1 : M	Guest: Partial / Booking: Total	A guest may have many bookings; every booking belongs to one
Room — Booking	1 : M	Room: Partial / Booking: Total	A room may have many bookings over time; every booking uses
Booking — Payment	1 : M	Booking: Partial / Payment: Total	A booking may have multiple payments; every payment belongs

Entity key structure: Hotel(hotel_id), Room(room_id), Guest(guest_id), Booking(booking_id), Payment(payment_id). Room also has the composite candidate key (hotel_id, room_number).

3. Part B — Tables, Keys & Constraints

Table	Primary Key	Candidate Key (not PK)	Foreign Key(s)	Composite Key / Rule
Hotels	hotel_id	hotel_name	None	—
Rooms	room_id	(hotel_id, room_number)	hotel_id → Hotels.hotel_id	Prevents duplicate room numbers within the same h
Guests	guest_id	email	None	—
Bookings	booking_id	No additional candidate key specified	room_id → Rooms.room_id; guest_id → Guests.guest_id	—
Payments	payment_id	No additional candidate key specified	booking_id → Bookings.booking_id	—

3.1 Table definitions used

The following definitions reflect the implementation visible in the screenshots. SERIAL primary keys are used, while UNIQUE constraints implement the specified candidate keys.

```
CREATE TABLE hotels (  
    hotel_id SERIAL PRIMARY KEY,  
    hotel_name VARCHAR(100) UNIQUE NOT NULL,  
    city VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE rooms (  
    room_id SERIAL PRIMARY KEY,  
    hotel_id INT NOT NULL,  
    room_number VARCHAR(10) NOT NULL,  
    room_type VARCHAR(30),  
    price_per_night NUMERIC(8,2),  
    FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id),  
    UNIQUE (hotel_id, room_number)  
);  
  
CREATE TABLE guests (  
    guest_id SERIAL PRIMARY KEY,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,  
    guest_details JSONB  
);  
  
CREATE TABLE bookings (  
    booking_id SERIAL PRIMARY KEY,  
    room_id INT NOT NULL,  
    guest_id INT NOT NULL,  
    check_in_date DATE NOT NULL,  
    check_out_date DATE NOT NULL,  
    FOREIGN KEY (room_id) REFERENCES rooms(room_id),  
    FOREIGN KEY (guest_id) REFERENCES guests(guest_id)  
);  
  
CREATE TABLE payments (  
    payment_id SERIAL PRIMARY KEY,  
    booking_id INT NOT NULL,  
    amount NUMERIC(10,2),  
    payment_date DATE,  
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)  
);
```

4. Sample Data

The database was populated with the sample records specified for the project.

Hotels	hotel_id	hotel_name	city
	1	Grand Palace	Goa
	2	City Comfort Inn	Pune

Rooms	room_id	hotel_id	room_number	room_type	price/night
	1	1	101	Deluxe	4500.00
	2	1	102	Suite	7000.00
	3	2	201	Standard	2500.00

Guests	guest_id	email	loyalty_tier
	1	karan.m@mail.com	gold
	2	divya.s@mail.com	silver

Bookings	booking_id	room_id	guest_id	check-in	check-out
	1	1	1	2026-09-25	2026-09-28
	2	3	2	2026-09-20	2026-09-22

Payments	payment_id	booking_id	amount	payment_date
	1	1	13500.00	2026-09-25
	2	2	5000.00	2026-09-20

5. Part C — Queries & Results

Q1. LEFT JOIN — Find rooms that have never been booked

Query executed:

```
SELECT
    r.room_id, r.hotel_id, r.room_number,
    r.room_type, r.price_per_night
FROM rooms r
LEFT JOIN bookings b
    ON r.room_id = b.room_id
WHERE b.booking_id IS NULL;
```

Observed result: room_id 2, hotel_id 1, room 102, Suite, price 7000.00. The screenshot shows one unbooked room.

room_id	hotel_id	room_number	room_type	price_per_night
2	1	102	Suite	7000.00

Q2. JSONB — Find guests with gold loyalty tier

The JSONB operator ->> extracts the value of the loyalty_tier field as text.

```
SELECT
    guest_id, email, guest_details
FROM guests
WHERE guest_details ->> 'loyalty_tier' = 'gold';
```

Observed result: guest_id 1, karan.m@mail.com, with loyalty_tier = gold.

Q3. Transaction / ACID / SAVEPOINT

The screenshots show the overlap check returning zero rows for the requested dates on room 2. The subsequent demonstration attempted to insert a booking with **booking_id = 1**, but that ID already existed in the database. PostgreSQL therefore raised a duplicate-key error and marked the transaction as aborted. Because the transaction had already failed before the SAVEPOINT was successfully established, the later ROLLBACK TO SAVEPOINT command reported that the savepoint did not exist.

Observed error: duplicate key value violates unique constraint "bookings_pkey"; Key (booking_id)=(1) already exists.

This is important evidence for the report: the final screenshots show that the original two bookings and two payments remained in the database.

Corrected reference transaction pattern

The following version avoids manually reusing an existing booking_id and demonstrates the intended SAVEPOINT behaviour. It is a reference implementation based on the project requirement; the screenshots document the earlier attempted run.

```
BEGIN;

-- Check for an overlapping booking for room 2
SELECT *
FROM bookings
WHERE room_id = 2
      AND check_in_date < DATE '2026-09-28'
      AND check_out_date > DATE '2026-09-25';

-- Insert booking only after the overlap check returns no rows
INSERT INTO bookings (room_id, guest_id, check_in_date, check_out_date)
VALUES (2, 1, DATE '2026-09-25', DATE '2026-09-28')
RETURNING booking_id;

SAVEPOINT before_payment;

-- Room 2 = 7000/night; 3 nights = 21000.
-- This payment is intentionally incorrect for SAVEPOINT demonstration.
INSERT INTO payments (booking_id, amount, payment_date)
VALUES (3, 10000, CURRENT_DATE);

-- Amount mismatch: rollback only the payment.
ROLLBACK TO SAVEPOINT before_payment;

-- Booking remains in the transaction.
COMMIT;
```

Expected logic: $7000 \times 3 \text{ nights} = 21000$. A payment of 10000 is therefore invalid for this demonstration and is rolled back to the SAVEPOINT, while the booking remains committed.

Q4. Date/Time — Length of stay and current-month bookings

The screenshots show the stay length calculated by subtracting check_in_date from check_out_date.

```
SELECT
    booking_id,
    check_in_date,
    check_out_date,
    check_out_date - check_in_date AS length_of_stay
FROM bookings;
```

booking_id	check_in_date	check_out_date	length_of_stay
1	2026-09-25	2026-09-28	3
2	2026-09-20	2026-09-22	2

Current-month filter used:

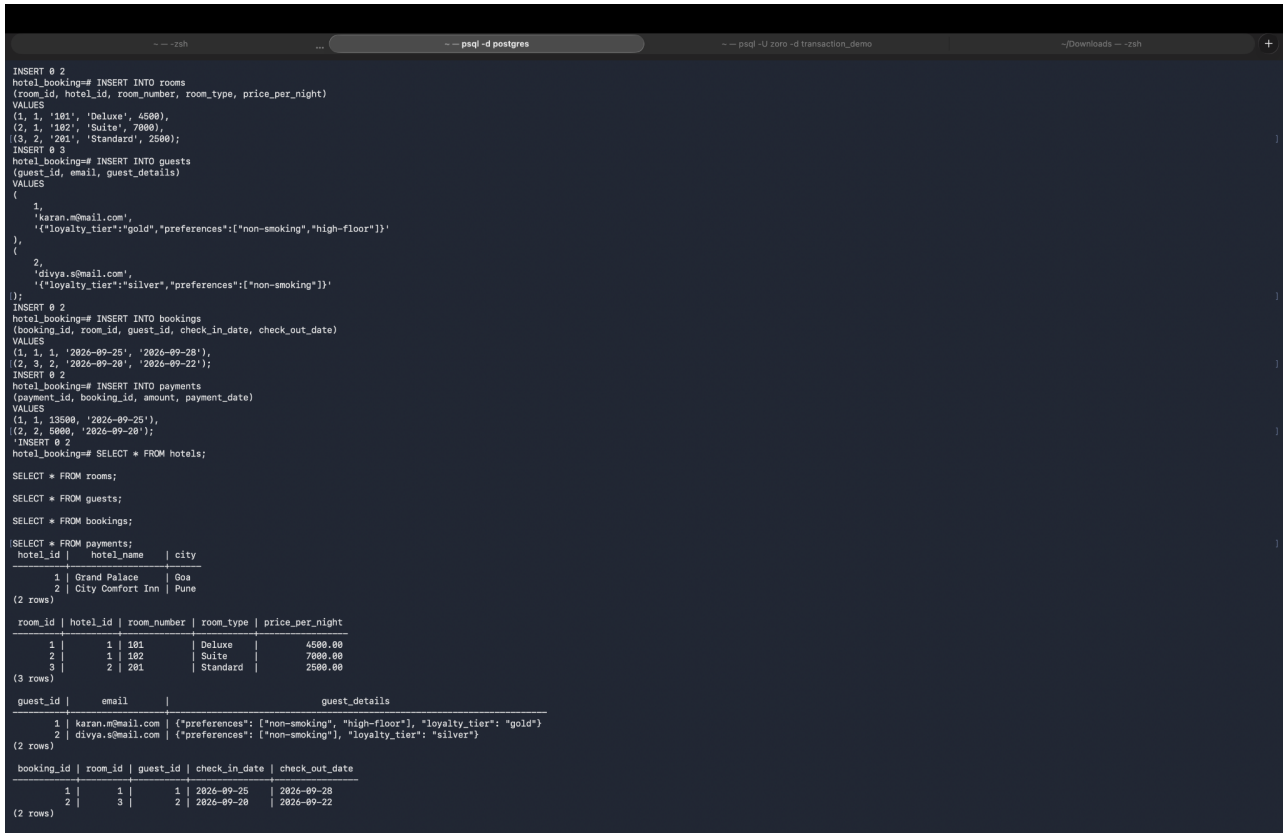
```
SELECT
    booking_id, room_id, guest_id,
    check_in_date, check_out_date
FROM bookings
WHERE check_in_date >= date_trunc('month', CURRENT_DATE)
    AND check_in_date < date_trunc('month', CURRENT_DATE)
    + INTERVAL '1 month';
```

Observed result: both existing bookings were returned because their check-in dates are in September 2026, the month represented by the captured execution session.

6. Execution Evidence — Terminal Screenshots

The following screenshots are included as submitted evidence from the PostgreSQL/psql execution. They are reproduced from the images provided with the project request.

Screenshot 1



```
INSERT 0 2
hotel_booking=# INSERT INTO rooms
(room_id, hotel_id, room_number, room_type, price_per_night)
VALUES
(1, 1, '101', 'Deluxe', 4500),
(2, 1, '102', 'Suite', 7000),
(3, 2, '201', 'Standard', 2500);
INSERT 0 3
hotel_booking=# INSERT INTO guests
(guest_id, email, guest_details)
VALUES
(
1,
'karan.s@gmail.com',
('loyalty_tier':'gold','preferences':['non-smoking','high-floor'])
),
(
2,
'diyya.s@gmail.com',
('loyalty_tier':'silver','preferences':['non-smoking'])
);
INSERT 0 2
hotel_booking=# INSERT INTO bookings
(booking_id, room_id, guest_id, check_in_date, check_out_date)
VALUES
(1, 1, 1, '2026-09-25', '2026-09-28'),
(2, 3, 2, '2026-09-29', '2026-09-22');
INSERT 0 2
hotel_booking=# INSERT INTO payments
(payment_id, booking_id, amount, payment_date)
VALUES
(1, 1, 15000, '2026-09-25'),
(2, 2, 5000, '2026-09-28');
INSERT 0 2
hotel_booking=# SELECT * FROM hotels;

SELECT * FROM rooms;

SELECT * FROM guests;

SELECT * FROM bookings;

SELECT * FROM payments;
 hotel_id | hotel_name | city
-----+-----+-----
      1 | Grand Palace | Goa
      2 | City Comfort Inn | Pune
(2 rows)

 room_id | hotel_id | room_number | room_type | price_per_night
-----+-----+-----+-----+-----
      1 |      1 |      101 | Deluxe | 4500.00
      2 |      1 |      102 | Suite | 7000.00
      3 |      2 |      201 | Standard | 2500.00
(3 rows)

 guest_id | email | guest_details
-----+-----+-----
      1 | karan.s@gmail.com | {'preferences': ['non-smoking', 'high-floor'], 'loyalty_tier': 'gold'}
      2 | diyya.s@gmail.com | {'preferences': ['non-smoking'], 'loyalty_tier': 'silver'}
(2 rows)

 booking_id | room_id | guest_id | check_in_date | check_out_date
-----+-----+-----+-----+-----
      1 |      1 |      1 | 2026-09-25 | 2026-09-28
      2 |      3 |      2 | 2026-09-29 | 2026-09-22
(2 rows)
```

Database creation, table creation, sample data and SELECT verification.

Screenshot 2

```
-- zsh
-- psql -d postgres
-- psql -U zoro -d transaction_demo
~/Downloads -- zsh

transaction_demo=# CREATE DATABASE hotel_booking;
CREATE DATABASE
transaction_demo=# \c hotel_booking
You are now connected to database "hotel_booking" as user "zoro".
hotel_booking=#
hotel_booking=# CREATE TABLE hotels (
  hotel_id SERIAL PRIMARY KEY,
  hotel_name VARCHAR(100) UNIQUE NOT NULL,
  city VARCHAR(50) NOT NULL
);
CREATE TABLE
hotel_booking=# CREATE TABLE rooms (
  room_id SERIAL PRIMARY KEY,
  hotel_id INT NOT NULL,
  room_number VARCHAR(10) NOT NULL,
  room_type VARCHAR(50),
  price_per_night NUMERIC(8,2),
  FOREIGN KEY (hotel_id)
    REFERENCES hotels(hotel_id),
  UNIQUE (hotel_id, room_number)
);
CREATE TABLE
hotel_booking=# UNIQUE (hotel_id, room_number)
hotel_booking=# CREATE TABLE guests (
  guest_id SERIAL PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  guest_details JSONB
);
ERROR: syntax error at or near "UNIQUE"
LINE 1: UNIQUE (hotel_id, room_number)
         ^
hotel_booking=# CREATE TABLE guests (
  guest_id SERIAL PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  guest_details JSONB
);
CREATE TABLE
hotel_booking=# CREATE TABLE bookings (
  booking_id SERIAL PRIMARY KEY,
  room_id INT NOT NULL,
  guest_id INT NOT NULL,
  check_in_date DATE NOT NULL,
  check_out_date DATE NOT NULL,
  FOREIGN KEY (room_id)
    REFERENCES rooms(room_id),
  FOREIGN KEY (guest_id)
    REFERENCES guests(guest_id)
);
CREATE TABLE
hotel_booking=# CREATE TABLE payments (
  payment_id SERIAL PRIMARY KEY,
  booking_id INT NOT NULL,
  amount NUMERIC(10,2),
  payment_date DATE,
  FOREIGN KEY (booking_id)
    REFERENCES bookings(booking_id)
);
CREATE TABLE
hotel_booking=# INSERT INTO hotels (hotel_id, hotel_name, city)
VALUES
(1, 'Grand Palace', 'Goa'),
(2, 'City Comfort Inn', 'Pune');
INSERT 0 2
```

Initial database/table creation attempt; the standalone UNIQUE statement error is visible.

Screenshot 3

```
-- zsh
-- psql -d postgres
-- psql -U zero -d transaction_demo
~/Downloads -- zsh

payment_id | booking_id | amount | payment_date
-----
1 | 1 | 13500.00 | 2026-09-25
2 | 2 | 5000.00 | 2026-09-28
(2 rows)

hotel_booking=# SELECT
r.room_id,
r.hotel_id,
r.room_number,
r.room_type,
r.price_per_night
FROM rooms r
LEFT JOIN bookings b
ON r.room_id = b.room_id
WHERE b.booking_id IS NULL;
 room_id | hotel_id | room_number | room_type | price_per_night
-----
2 | 1 | 102 | Suite | 7000.00
(1 row)

hotel_booking=# SELECT
guest_id,
email,
guest_details
FROM guests
WHERE guest_details --> 'loyalty_tier' = 'gold';
 guest_id | email | guest_details
-----
1 | karan.m@mail.com | {"preferences": ["non-smoking", "high-floor"], "loyalty_tier": "gold"}
(1 row)

hotel_booking=# BEGIN;
-- 1. Check for overlapping booking
SELECT *
FROM bookings
WHERE room_id = 2
AND check_in_date < DATE '2026-09-28'
AND check_out_date > DATE '2026-09-25';
 booking_id | room_id | guest_id | check_in_date | check_out_date
-----
(0 rows)

hotel_booking=# BEGIN;
-- 1. Check for overlapping booking
SELECT *
FROM bookings
WHERE room_id = 2
AND check_in_date < DATE '2026-09-28'
AND check_out_date > DATE '2026-09-25';
WARNING: there is already a transaction in progress
 booking_id | room_id | guest_id | check_in_date | check_out_date
-----
(0 rows)

hotel_booking=# INSERT INTO bookings
(room_id, guest_id, check_in_date, check_out_date)
VALUES
(2, 1, '2026-09-25', '2026-09-28');
RETURNING booking_id;
ERROR: duplicate key value violates unique constraint "bookings_pkey"
DETAIL: Key (booking_id)=(1) already exists.
hotel_booking=# SAVEPOINT before_payment;
```

Execution of joins, JSONB query, transaction start, overlap check and booking insertion attempt.

Screenshot 4

```
-- zsh
-- psql -d postgres
-- psql -U zero -d transaction_demo
~/Downloads -- zsh

(booking_id, amount, payment_date)
VALUES
(9, 10000, CURRENT_DATE);
ERROR: current transaction is aborted, commands ignored until end of transaction block
hotel_booking=# ROLLBACK TO SAVEPOINT before_payment;
ERROR: savepoint "before_payment" does not exist
hotel_booking=# COMMIT;
ROLLBACK
hotel_booking=# SELECT * FROM bookings;

SELECT * FROM payments;
booking_id | room_id | guest_id | check_in_date | check_out_date
-----
1 | 1 | 1 | 2026-09-25 | 2026-09-28
2 | 3 | 2 | 2026-09-20 | 2026-09-22
(2 rows)

payment_id | booking_id | amount | payment_date
-----
1 | 1 | 13500.00 | 2026-09-25
2 | 2 | 5000.00 | 2026-09-20
(2 rows)

hotel_booking=# SELECT
booking_id,
check_in_date,
check_out_date,
check_out_date - check_in_date AS length_of_stay
FROM bookings;
booking_id | check_in_date | check_out_date | length_of_stay
-----
1 | 2026-09-25 | 2026-09-28 | 3
2 | 2026-09-20 | 2026-09-22 | 2
(2 rows)

hotel_booking=# SELECT
booking_id,
room_id,
guest_id,
check_in_date,
check_out_date
FROM bookings
WHERE check_in_date >= date_trunc('month', CURRENT_DATE)
AND check_in_date < date_trunc('month', CURRENT_DATE) + INTERVAL '1 month';
booking_id | room_id | guest_id | check_in_date | check_out_date
-----
1 | 1 | 1 | 2026-09-25 | 2026-09-28
2 | 3 | 2 | 2026-09-20 | 2026-09-22
(2 rows)

hotel_booking=#
hotel_booking=#
-- =====
-- WITH PROJECT 5
-- HOTEL BOOKING SYSTEM
-- =====

-- Create database separately:
-- CREATE DATABASE hotel_booking;

-- Connect:
-- \c hotel_booking

-- =====
-- PART B: TABLE CREATION
-- =====

CREATE TABLE hotels (
```

Transaction/output verification, length-of-stay query and current-month query.

Screenshot 5

```
-- zsh
-- psql -d postgres
-- psql -U zero -d transaction_demo
~/Downloads -- zsh

CREATE TABLE hotels (
    hotel_id SERIAL PRIMARY KEY,
    hotel_name VARCHAR(100) UNIQUE NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE rooms (
    room_id SERIAL PRIMARY KEY,
    hotel_id INT NOT NULL,
    room_number VARCHAR(10) NOT NULL,
    room_type VARCHAR(30),
    price_per_night NUMERIC(8,2),

    FOREIGN KEY (hotel_id)
        REFERENCES hotels(hotel_id),

    UNIQUE (hotel_id, room_number)
);

CREATE TABLE guests (
    guest_id SERIAL PRIMARY KEY,
    email VARCHAR(100) UNIQUE NOT NULL,
    guest_details JSONB
);

CREATE TABLE bookings (
    booking_id SERIAL PRIMARY KEY,
    room_id INT NOT NULL,
    guest_id INT NOT NULL,
    check_in_date DATE NOT NULL,
    check_out_date DATE NOT NULL,

    FOREIGN KEY (room_id)
        REFERENCES rooms(room_id),

    FOREIGN KEY (guest_id)
        REFERENCES guests(guest_id)
);

CREATE TABLE payments (
    payment_id SERIAL PRIMARY KEY,
    booking_id INT NOT NULL,
    amount NUMERIC(10,2),
    payment_date DATE,

    FOREIGN KEY (booking_id)
        REFERENCES bookings(booking_id)
);

--
-- SAMPLE DATA
--

INSERT INTO hotels
(hotel_id, hotel_name, city)
VALUES
(1, 'Grand Palace', 'Goa'),
(2, 'City Comfort Inn', 'Pune');

INSERT INTO rooms
(room_id, hotel_id, room_number, room_type, price_per_night)
VALUES
(1, 1, '101', 'Deluxe', 4500),
(2, 1, '102', 'Suite', 7000),
(3, 2, '201', 'Standard', 2500);

INSERT INTO guests
```

Schema and sample-data script shown in the terminal.

Screenshot 6

```
-- zsh
-- psql -d postgres
-- psql -U zero -d transaction_demo
~/Downloads -- zsh

INSERT INTO guests
(guest_id, email, guest_details)
VALUES
(
1,
'karan.m@gmail.com',
'{"loyalty_tier":"gold","preferences":["non-smoking","high-floor"]}'
),
(
2,
'divya.s@gmail.com',
'{"loyalty_tier":"silver","preferences":["non-smoking"]}'
);

INSERT INTO bookings
(booking_id, room_id, guest_id, check_in_date, check_out_date)
VALUES
(1, 1, 1, '2026-09-25', '2026-09-28'),
(2, 3, 2, '2026-09-28', '2026-09-22');

INSERT INTO payments
(payment_id, booking_id, amount, payment_date)
VALUES
(1, 1, 13500, '2026-09-25'),
(2, 2, 5000, '2026-09-28');

-- =====
-- Q1: LEFT JOIN - UNBOOKED ROOMS
-- =====

SELECT
r.room_id,
r.hotel_id,
r.room_number,
r.room_type,
r.price_per_night
FROM rooms r
LEFT JOIN bookings b
ON r.room_id = b.room_id
WHERE b.booking_id IS NULL;

-- =====
-- Q2: JSQB - GOLD GUESTS
-- =====

SELECT
guest_id,
email,
guest_details
FROM guests
WHERE guest_details -->> 'loyalty_tier' = 'gold';

-- =====
-- Q3: TRANSACTION + SAVEPOINT
-- =====

BEGIN;

-- Check overlapping booking
SELECT *
FROM bookings
WHERE room_id = 2
AND check_in_date < DATE '2026-09-28'
AND check_out_date > DATE '2026-09-25';
```

Sample-data insertion followed by Q1 and Q2 query sections.

Screenshot 7

```
-- zsh
-- psql -d postgres
-- psql -U zero -d transaction_demo
~/Downloads -- zsh

-- Insert booking if no rows are returned above
INSERT INTO bookings
(room_id, guest_id, check_in_date, check_out_date)
VALUES
(2, 1, '2026-09-28', '2026-09-28')
RETURNING booking_id;

-- Savepoint before payment
SAVEPOINT before_payment;

-- Incorrect payment for demonstration
INSERT INTO payments
(booking_id, amount, payment_date)
VALUES
(3, 10000, CURRENT_DATE);

-- Expected:
-- 7000 * 3 = 21000
-- Actual payment = 10000
-- Therefore rollback only payment

ROLLBACK TO SAVEPOINT before_payment;

-- Booking remains
COMMIT;

=====
-- Q4: LENGTH OF STAY
=====

SELECT
    booking_id,
    check_in_date,
    check_out_date,
    check_out_date - check_in_date AS length_of_stay
FROM bookings;

=====
-- Q4: CURRENT MONTH BOOKINGS
=====

SELECT
    booking_id,
    room_id,
    guest_id,
    check_in_date,
    check_out_date
FROM bookings
WHERE check_in_date >= date_trunc('month', CURRENT_DATE)
AND check_in_date < date_trunc('month', CURRENT_DATE) + INTERVAL '1 month';
ERROR:  relation "hotels" already exists
ERROR:  relation "rooms" already exists
ERROR:  relation "guests" already exists
ERROR:  relation "bookings" already exists
ERROR:  relation "payments" already exists
ERROR:  duplicate key value violates unique constraint "hotels_pkey"
DETAIL:  Key (hotel_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "rooms_pkey"
DETAIL:  Key (room_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "guests_pkey"
DETAIL:  Key (guest_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "bookings_pkey"
DETAIL:  Key (booking_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "payments_pkey"
DETAIL:  Key (payment_id)=(1) already exists.
```

Q3 transaction/SAVEPOINT section and Q4 date/time queries.

Screenshot 8

```
-- zsh
-- psql -d postgres
-- psql -U zero -d transaction_demo
~/Downloads -- zsh

check_in_date,
check_out_date,
check_out_date - check_in_date AS length_of_stay
FROM bookings;

-- =====
-- Q4: CURRENT MONTH BOOKINGS
-- =====

SELECT
  booking_id,
  room_id,
  guest_id,
  check_in_date,
  check_out_date
FROM bookings
WHERE check_in_date >= date_trunc('month', CURRENT_DATE)
AND check_in_date < date_trunc('month', CURRENT_DATE) + INTERVAL '1 month';
ERROR:  relation "hotels" already exists
ERROR:  relation "rooms" already exists
ERROR:  relation "guests" already exists
ERROR:  relation "bookings" already exists
ERROR:  relation "payments" already exists
ERROR:  duplicate key value violates unique constraint "hotels_pkey"
DETAIL:  Key (hotel_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "rooms_pkey"
DETAIL:  Key (room_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "guests_pkey"
DETAIL:  Key (guest_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "bookings_pkey"
DETAIL:  Key (booking_id)=(1) already exists.
ERROR:  duplicate key value violates unique constraint "payments_pkey"
DETAIL:  Key (payment_id)=(1) already exists.
 room_id | hotel_id | room_number | room_type | price_per_night
-----+-----+-----+-----+-----
      2 |         |      1 | 102       | Suite          |      7000.00
(1 row)

guest_id | email | guest_details
-----+-----+-----
      1 | karan.m@gmail.com | {"preferences": ["non-smoking", "high-floor"], "loyalty_tier": "gold"}
(1 row)

BEGIN
 booking_id | room_id | guest_id | check_in_date | check_out_date
-----+-----+-----+-----+-----
(0 rows)

ERROR:  duplicate key value violates unique constraint "bookings_pkey"
DETAIL:  Key (booking_id)=(2) already exists.
ERROR:  current transaction is aborted, commands ignored until end of transaction block
ERROR:  current transaction is aborted, commands ignored until end of transaction block
ERROR:  savepoint "before_payment" does not exist
ROLLBACK
 booking_id | check_in_date | check_out_date | length_of_stay
-----+-----+-----+-----
      1 | 2026-09-25    | 2026-09-28    | 3
      2 | 2026-09-28    | 2026-09-22    | 2
(2 rows)

 booking_id | room_id | guest_id | check_in_date | check_out_date
-----+-----+-----+-----+-----
      1 |      1 |         | 1 | 2026-09-25    | 2026-09-28
      2 |      3 |         | 2 | 2026-09-28    | 2026-09-22
(2 rows)

hotel_booking=#
```

Final terminal state showing duplicate-key transaction errors and resulting query outputs.

7. Conclusion

The Mini Project 5 implementation demonstrates the requested hotel booking database using PostgreSQL. The schema establishes primary keys, candidate keys, foreign keys, a composite candidate key for room identification within a hotel, JSONB guest details, relational joins, and date calculations. The screenshots provide execution evidence for the created database and the required query outputs.

The transaction section also documents an execution issue caused by manually reusing an existing primary-key value. The corrected reference pattern above uses PostgreSQL-generated booking IDs and places the payment operation after a SAVEPOINT, which matches the intended ACID/SAVEPOINT requirement.

Group 9 - Mini Project 5

Himanshu Katiyara • Asad Jamil Antule • Sahil Kumar • Sasanka
SekharKundu